COMPUTATIONAL FLUID DYNAMICS USING PHYSICS-INFORMED NEURAL NETWORKS

A Comparative Study of Numerical and Machine Learning Approaches for Flow Past a Cylinder at Re = 100 to 30,000

UGP Project Report

Submitted by

Harsh Pratap Singh (230447) and Sanket Bansal (230921)

Department of Civil Engineering

Indian Institute of Technology Kanpur

April 2026

Abstract

This report presents a comprehensive two-phase study of incompressible fluid flow past a circular cylinder across a range of Reynolds numbers (Re = 100, 1,000, 10,000, and 30,000). The first phase establishes a high-fidelity numerical baseline using OpenFOAM, an open-source Computational Fluid Dynamics (CFD) platform implementing the Finite Volume Method. The second phase implements a Physics-Informed Neural Network (PINN) framework using the DeepXDE library and PyTorch to replicate and extend the CFD results using a data-driven, mesh-free approach.

In the CFD phase, both laminar simulations (Re = 100 and 1,000) and turbulent RANS simulations with the k–ω SST model (Re = 10,000 and 30,000) were performed using the pimpleFoam transient solver with the PIMPLE pressure-velocity coupling algorithm. The resulting flow fields, including velocity, pressure, turbulent kinetic energy (k), and specific dissipation rate (ω), serve as ground truth for the subsequent PINN training.

In the PINN phase, the Re = 10,000 steady flow field is reconstructed from sparse OpenFOAM data (approximately 2% of interior points) supplemented by dense boundary patch data. The network architecture employs a Parallel Fully Connected Network (PFNN) with Fourier feature encoding, multi-scale input normalization, log-space viscosity parameterization, and a three-stage training strategy comprising physics warm-up, combined data-physics training, and L-BFGS fine-tuning. Quantitative results demonstrate relative L2 errors of 2.91% for streamwise velocity and 2.26% for pressure, with 14.82% for the cross-stream velocity component. The report discusses model limitations, applied improvements, and future research directions.

1. Introduction

1.1 Background and Motivation

Flow past a bluff body such as a circular cylinder is one of the most fundamental and extensively studied problems in fluid mechanics. Its rich phenomenology - ranging from laminar steady flow at low Reynolds numbers through periodic vortex shedding to fully turbulent wakes - makes it an ideal benchmark for evaluating both numerical methods and emerging machine learning approaches. Practical engineering relevance spans offshore structures, heat exchangers, bridge piers, overhead transmission lines, and aerodynamic drag minimization.

Traditional Computational Fluid Dynamics tools such as OpenFOAM have been the industry standard for simulating such flows for several decades. While these methods are accurate and well-validated, they require significant computational infrastructure: structured or unstructured mesh generation, iterative linear solvers scaled to millions of degrees of freedom, and dedicated high-performance computing resources. For turbulent flows at high Reynolds numbers, even moderate-resolution meshes can require hours to days of computation.

Physics-Informed Neural Networks (PINNs) represent a fundamentally different computational paradigm. By embedding the governing partial differential equations into the neural network loss function, PINNs can approximate solutions that are simultaneously consistent with available data and physical laws, without requiring a computational mesh. This approach holds particular promise for scenarios involving sparse sensor data, inverse problems where boundary conditions are partially unknown, and real-time digital twin applications requiring rapid inference.

This project implements both approaches in sequence: first establishing a robust CFD baseline across multiple Reynolds numbers using OpenFOAM, then leveraging that data to train and validate a PINN model for the turbulent Re = 10,000 case. The comparative analysis provides quantitative insight into the accuracy, computational cost, and practical viability of each approach.

1.2 Scope of the Study

The study is organized into two sequential phases:

Phase	Tool	Re Range	Flow Regime	Purpose
Phase 1: CFD	OpenFOAM	100, 1000, 10,000, 30,000	Laminar → Turbulent	Generate ground truth data
Phase 2: PINN	DeepXDE + PyTorch	10,000 (focus)	Turbulent (RANS regime)	Reconstruct from sparse data

1.3 Objectives

1. To perform steady and transient incompressible flow simulations past a circular cylinder at Re = 100, 1,000, 10,000, and 30,000 using OpenFOAM.
2. To implement appropriate turbulence models (direct laminar for Re ≤ 1,000; k–ω SST RANS for Re ≥ 10,000) and validate against established literature.
3. To extract structured datasets from OpenFOAM simulation outputs for each boundary patch and sparse interior region.
4. To implement an optimized PINN using DeepXDE with embedded 2D incompressible Navier-Stokes equations as physics constraints.
5. To apply state-of-the-art training improvements including Fourier feature encoding, PFNN architecture, multi-stage training, and log-viscosity parameterization.
6. To quantitatively evaluate PINN prediction accuracy against OpenFOAM ground truth using relative L2 error metrics.
7. To critically assess the strengths and limitations of both approaches and outline future improvements.

2. Governing Equations

2.1 Incompressible Navier-Stokes Equations

The motion of an incompressible Newtonian fluid is governed by the Navier-Stokes equations, which represent conservation of mass and linear momentum. In their general time-dependent form:

2.1.1 Continuity Equation (Conservation of Mass)

For an incompressible fluid (ρ = constant), the divergence of the velocity field is identically zero:

$$\nabla \cdot u = 0$$

where u = (u, v) is the two-dimensional velocity field with components u (streamwise) and v (cross-stream).

2.1.2 Momentum Equation (Conservation of Momentum)

The unsteady, incompressible momentum equation in vector form is:

$$\partial u/\partial t + (u \cdot \nabla)u = -(1/\rho)\nabla p + \nu \nabla 2u$$

For steady-state simulations (RANS and the PINN formulation), the ∂u/∂t term vanishes, yielding:

$$(u \cdot \nabla)u = -(1/\rho)\nabla p + \nu \nabla 2u$$

Expanded in Cartesian coordinates for 2D flow:

$$u(\partial u/\partial x) + v(\partial u/\partial y) = -\partial p/\partial x + \nu(\partial^2 u/\partial x^2 + \partial^2 u/\partial y^2) \quad [x\text{-momentum}]$$
$$u(\partial v/\partial x) + v(\partial v/\partial y) = -\partial p/\partial y + \nu(\partial^2 v/\partial x^2 + \partial^2 v/\partial y^2) \quad [y\text{-momentum}]$$

where p is the kinematic pressure (p/ρ), and ν is the kinematic viscosity.

2.2 Reynolds Number and Flow Regimes

The Reynolds number is the primary dimensionless parameter governing the transition between laminar and turbulent flow:

$$Re = U \cdot D / \nu$$

where U is the free-stream velocity, D is the cylinder diameter, and ν is the kinematic viscosity.

Re	Flow Regime	Key Flow Features	Simulation Approach
100	Laminar, steady	Symmetric attached wake, no separation	Direct NS solution
1,000	Laminar, unsteady	Periodic von Kármán vortex shedding begins	Direct NS solution
10,000	Turbulent	Wide turbulent wake, shear layers	k–ω SST RANS
30,000	Fully turbulent	Separated boundary layers, complex wake	k–ω SST RANS

2.3 RANS Turbulence Modeling

For Re = 10,000 and 30,000, the instantaneous flow contains turbulent fluctuations across a wide range of length and time scales that are computationally prohibitive to resolve directly. Reynolds-Averaged Navier-Stokes (RANS) modeling decomposes each flow variable into a mean part and a fluctuating part:

$$u = \bar{u} + u', \quad p = \bar{a} + p'$$

Substituting and time-averaging introduces the Reynolds stress tensor, which requires a turbulence closure model. The k–ω SST (Shear Stress Transport) model was selected for this study.

2.3.1 k–ω SST Transport Equations

Two additional transport equations are solved for turbulent kinetic energy k and specific dissipation rate ω:

$$\partial k/\partial t + u\partial k/\partial x^i = P\_k - \beta^* k\omega + \nabla\cdot[(v + \sigma\_k\, v\_t)\nabla k]$$

$$\partial\omega/\partial t + u\partial\omega/\partial x^i = \alpha\omega/k \cdot P\_k - \beta\omega^2 + \nabla\cdot[(v + \sigma\_\omega\, v\_t)\nabla\omega] + 2(1-F_1)\sigma\_\omega 2/\omega\ \nabla k\cdot\nabla\omega$$

The turbulent (eddy) viscosity is then computed as:

$$v\_t = a_1 k \,/\, max(a_1\omega, S\cdot F_2)$$

The k–ω SST model blends the standard k–ω model (accurate near walls) with the k–ε model (accurate in free shear flows) using a blending function F_1, providing superior performance for flows with adverse pressure gradients and separation, as encountered at the cylinder surface.

3. Phase I - CFD Methodology (OpenFOAM)

3.1 Computational Domain and Geometry

The computational domain is a rectangular channel with a circular cylinder mounted perpendicular to the flow. The domain dimensions were chosen to minimize blockage effects and ensure fully developed inflow and adequate wake development:

- Cylinder diameter: D = 0.1 m (reference length scale)
- Domain extents: x ∈ [−5D, +10D] (streamwise), y ∈ [−5D, +5D] (cross-stream)
- Spanwise extent (quasi-2D): z ∈ [0, 0.01 m] (one cell, effectively 2D)
- Cylinder center: origin (0, 0)
- Total blockage ratio: D / (2 × 5D) = 10% (standard for cylinder benchmark)

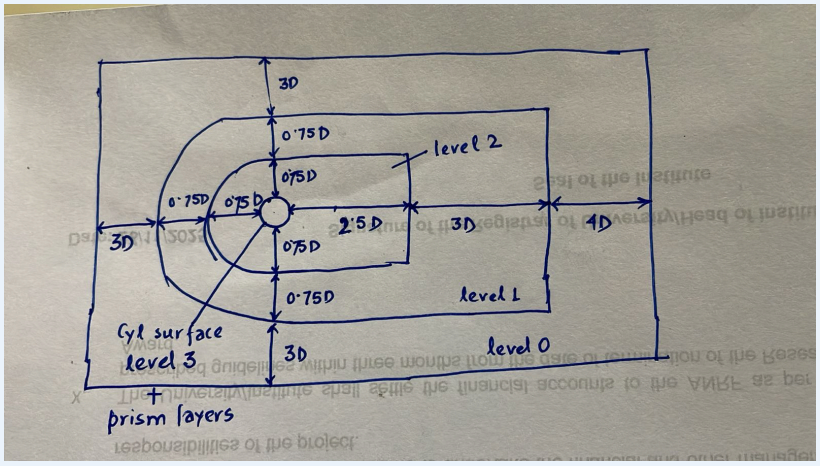


Figure 3.1 - Computational domain schematic showing cylinder position, boundary patch labels, and key dimensions. (D = cylinder diameter)

3.2 Mesh Generation

The computational mesh was generated using OpenFOAM's built-in meshing utilities in a three-step process:

3.2.1 blockMesh: Base Structured Mesh

A background hexahedral block mesh was created using blockMesh, providing a uniform base mesh across the entire rectangular domain. The block structure ensures predominantly hexahedral cells throughout the domain, which provide better numerical accuracy and convergence compared to tetrahedral elements.

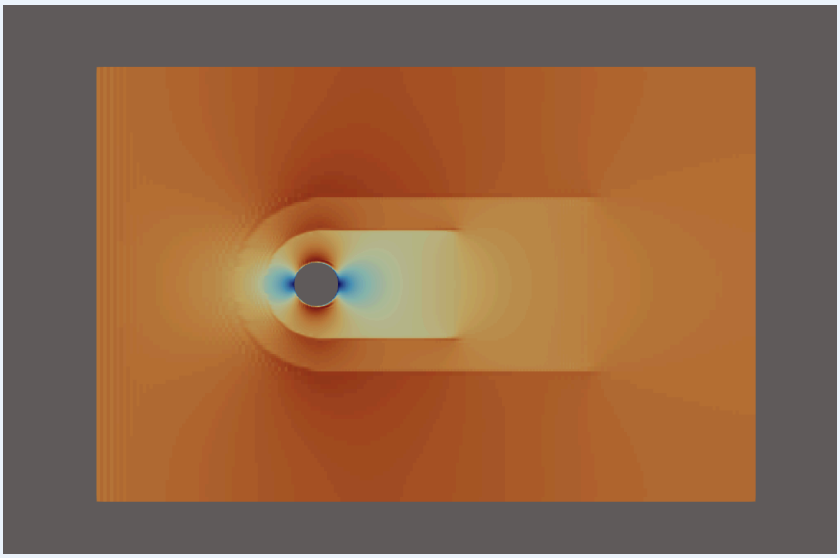
3.2.2 surfaceFeatureExtract: Geometry Feature Detection

Feature edges on the cylinder surface geometry were extracted using surfaceFeatureExtract. These feature edges guide snappyHexMesh in correctly capturing sharp geometric features and ensuring a smooth surface mesh on the curved cylinder wall.

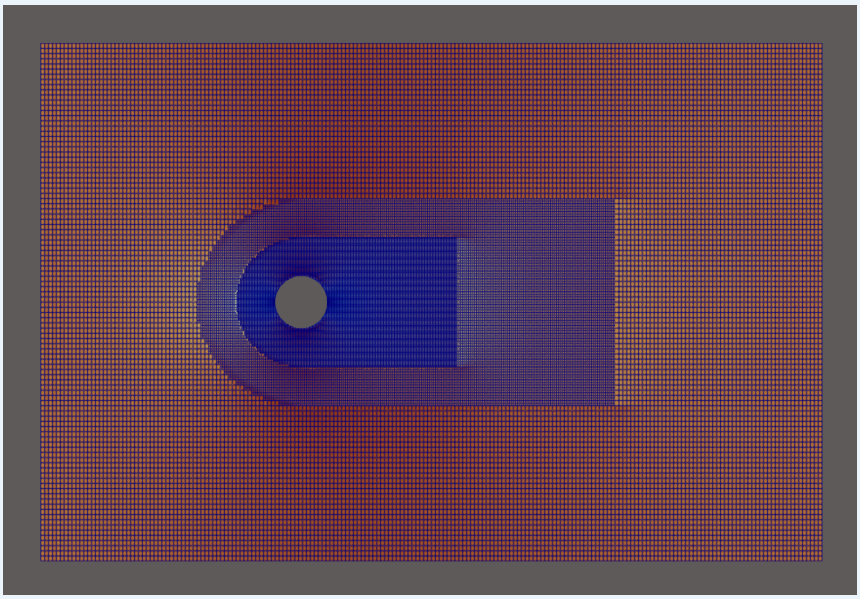
3.2.3 snappyHexMesh: Surface Snapping and Boundary Layer Refinement

The three-step snappyHexMesh process (castellating, snapping, and layer addition) was used to:

- Conformally snap the base mesh to the cylinder surface geometry
- Refine mesh cells in the near-cylinder region and wake zone
- Add structured prismatic boundary layer cells on the cylinder wall to resolve the viscous sublayer
- Maintain mesh quality metrics (non-orthogonality, skewness, aspect ratio) within acceptable tolerances



(a)



(b)

Figure 3.2 - Computational mesh: (a) full domain view showing boundary layer region near cylinder; (b) close-up of mesh refinement around cylinder wall.

3.2.4 Mesh Quality Summary

Mesh Property	Value / Criterion
Cell type	Predominantly hexahedral
Near-wall refinement	Boundary layer prisms on cylinder surface
Max non-orthogonality	< 60° (within OpenFOAM recommended limit)
Max skewness	< 4 (acceptable for numerical stability)
First cell y+ (RANS cases)	Targeted y+ ≈ 1 for SST near-wall treatment

3.3 Boundary Conditions

Boundary Patch	Velocity (U)	Pressure (p)	k (RANS)	ω (RANS)
Inlet	Uniform u = 10 m/s, v = 0	Zero gradient	Estimated from I, L	Estimated from I, L
Outlet	Zero gradient	p = 0 (reference)	Zero gradient	Zero gradient
Cylinder wall	No-slip: u = v = 0	Zero gradient	Wall function	omegaWallFunction

Boundary Patch	Velocity (U)	Pressure (p)	k (RANS)	ω (RANS)
Top boundary	Symmetry (or slip)	Zero gradient	Symmetry	Symmetry
Bottom boundary	Symmetry (or slip)	Zero gradient	Symmetry	Symmetry
Front/Back (z)	Empty (2D)	Empty (2D)	Empty (2D)	Empty (2D)

For RANS cases, the turbulent boundary conditions at the inlet are estimated from assumed turbulence intensity I = 5% and a turbulent length scale L based on the cylinder diameter:

$$k_{inlet} = (3/2)(U\infty \cdot I)^2, \quad \omega_{inlet} = k^{0.5} / (C_\mu^{0.25} \cdot L)$$

3.4 Numerical Solver and Algorithm

3.4.1 Solver: pimpleFoam

All simulations were performed using the pimpleFoam solver, which is OpenFOAM's transient incompressible flow solver. pimpleFoam is selected over the steady-state simpleFoam because:

- At Re = 1,000, the flow is intrinsically unsteady (vortex shedding) and requires transient simulation
- At Re = 10,000 and 30,000, even RANS may exhibit slow transients before reaching the statistically steady state
- pimpleFoam supports larger time-steps than PISO via its multiple outer correction loops

3.4.2 PIMPLE Pressure-Velocity Coupling

The PIMPLE algorithm combines the SIMPLE (Semi-Implicit Method for Pressure-Linked Equations) and PISO (Pressure Implicit with Splitting of Operators) algorithms. At each time step, multiple outer correction loops are performed (controlled by nOuterCorrectors), each containing PISO inner loops (nCorrectors) for pressure-velocity coupling. This approach provides:

- Stability for large Courant numbers (Co > 1 is permissible with pimpleFoam)
- Better convergence at each time step for stiff turbulent flows
- Flexibility to relax under-relaxation factors between outer iterations

3.4.3 Discretization Schemes

Term	Scheme	Order	Notes
Time derivative (∂/∂t)	Euler implicit	1st order	Unconditionally stable, dissipative
Gradient (∇)	Gauss linear	2nd order	Standard central differencing
Divergence (convection)	Gauss linearUpwind / upwind	1st–2nd order	Upwind for stability at high Re
Laplacian (diffusion)	Gauss linear corrected	2nd order	Explicit non-orthogonality correction

Term	Scheme	Order	Notes
Interpolation	linear	2nd order	Cell-face interpolation

3.5 Simulation Control and Parallel Computation

Simulation control parameters were set in the controlDict file. Time-step selection was based on maintaining the Courant number below the stability limit. Domain decomposition was performed using OpenFOAM's decomposeParDict for parallel execution across multiple CPU cores:

- Decomposition method: scotch (graph-based, minimizes inter-processor boundaries)
- Convergence monitoring: residuals for p, U, k, ω tracked at each time step
- Data output: VTK format exported via foamToVTK for ParaView visualization

4. Phase I - CFD Results and Flow Analysis

4.1 Laminar Flow Regime (Re = 100)

At Re = 100, the flow is in the laminar, steady-separated regime. A pair of symmetric recirculation vortices forms immediately behind the cylinder. The wake length is finite and does not extend far downstream. Streamlines are smooth and symmetric about the cylinder centreline. No unsteadiness is observed in the converged steady solution.

Key flow features at Re = 100:

- Symmetric recirculation region attached to the rear of the cylinder
- Drag coefficient C_D ≈ 1.3–1.5 (consistent with established literature values)
- No vortex shedding; Strouhal number is not defined in steady laminar flow
- Pressure coefficient distribution shows smooth stagnation point at the leading edge

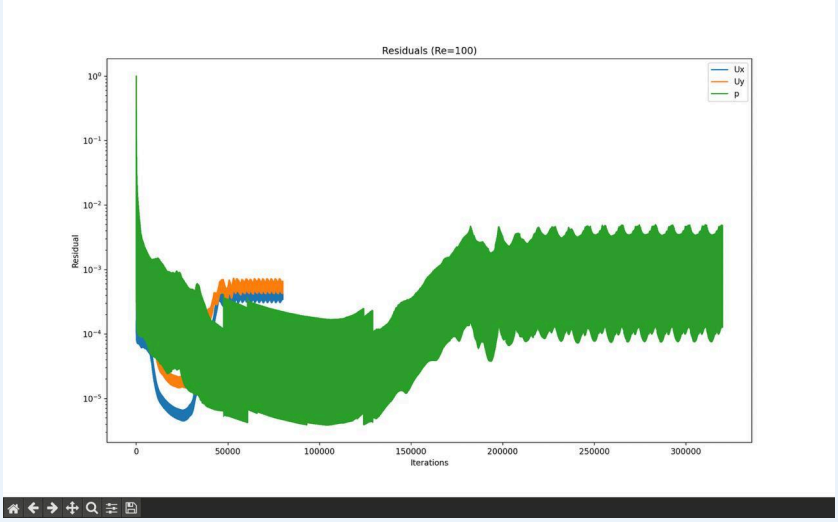


Figure 4.1 - Velocity magnitude and streamline contours for Re = 100. (a) Full domain view; (b) Close-up of symmetric recirculation region behind cylinder.

4.2 Laminar Unsteady Flow Regime (Re = 1,000)

At Re = 1,000, the wake becomes unsteady and the classical von Kármán vortex street develops. Alternating vortices of opposite sign are shed periodically from the upper and lower sides of the cylinder.

The Strouhal number St = fD/U∞ characterizes the dimensionless shedding frequency. At this Reynolds number, St ≈ 0.20–0.21, consistent with published experimental data.

Key flow features at Re = 1,000:

- Periodic vortex shedding with regular alternating street structure
- Significant velocity fluctuations in the near-wake (x/D < 5)
- Lift coefficient oscillates sinusoidally with shedding period
- Increased drag and pressure recovery compared to Re = 100 due to stronger wake interaction

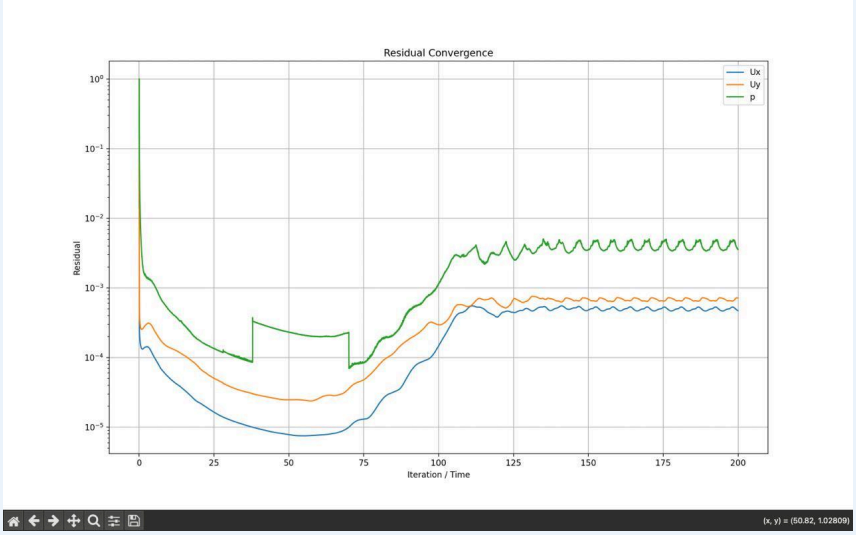


Figure 4.2 - Instantaneous vorticity contours and velocity field for Re = 1,000 showing von Kármán vortex street.

4.3 Turbulent Flow Regime (Re = 10,000)

At Re = 10,000, the flow is in the sub-critical turbulent regime. The k–ω SST RANS model captures the time-averaged turbulent flow field, including the turbulent wake and boundary layer separation. The RANS solution represents the ensemble-averaged mean flow, smearing out individual turbulent fluctuations.

Key flow features at Re = 10,000:

- Wide turbulent wake with significant velocity deficit persisting to x/D ≈ 10 and beyond
- Elevated turbulent kinetic energy k concentrated in two shear layers emanating from the cylinder shoulders
- Pressure coefficient C_p shows classic bluff body distribution with C_p,base ≈ −1.2 at rear stagnation

- Boundary layer separation occurs before the rear of the cylinder ($\theta_{sep} \approx 80–90°$ from stagnation)
- Effective turbulent viscosity ν_t up to 100× the laminar value in the wake core

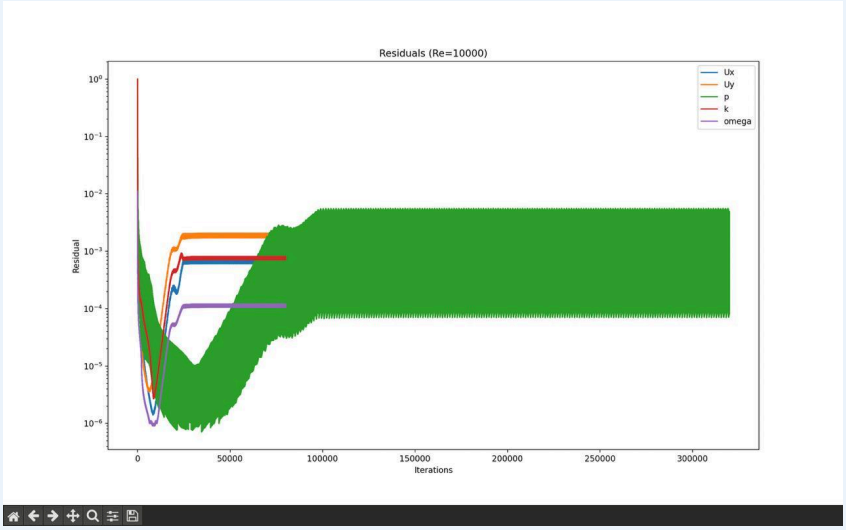


Figure 4.3 - Mean flow field for Re = 10,000 (RANS steady state)

4.4 Fully Turbulent Flow Regime (Re = 30,000)

At Re = 30,000, the flow exhibits stronger inertial effects and more aggressive boundary layer separation. The turbulent wake is wider and the velocity deficit is more pronounced compared to Re = 10,000. The k–ω SST model continues to provide physically reasonable RANS predictions, though at this Reynolds number the limitations of steady RANS become more apparent.

Key flow features at Re = 30,000:

- Broader turbulent wake compared to Re = 10,000
- Higher peak turbulent kinetic energy in shear layers
- Stronger pressure suction on the rear of the cylinder, lower base pressure $C_{p,base}$
- Steeper velocity gradients near the cylinder wall requiring finer boundary layer mesh resolution

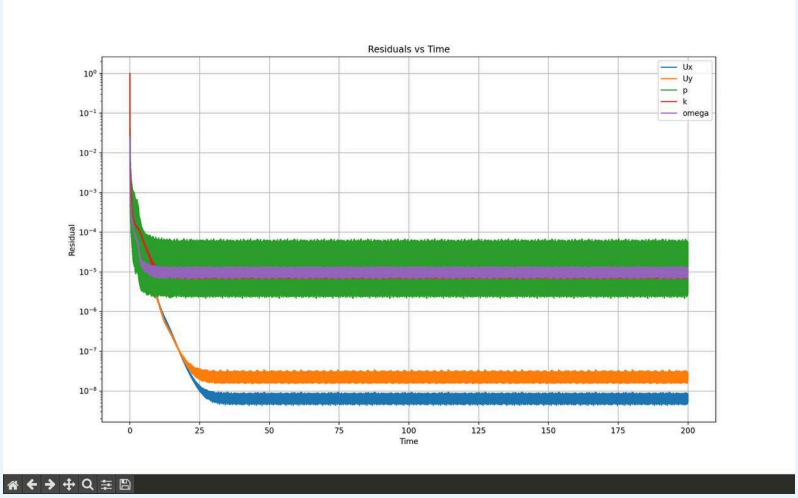


Figure 4.4 - Mean flow field for Re = 30,000 (RANS steady state)

4.5 Effect of Reynolds Number: Summary

Parameter	Re = 100	Re = 1,000	Re = 10,000	Re = 30,000
Flow regime	Laminar steady	Laminar unsteady	Turbulent RANS	Turbulent RANS
Vortex shedding	None	Regular (St ≈ 0.21)	Turbulent wake	Turbulent wake
Wake length	Finite, short	Extends to ~5D	Extends to >10D	Extends to >15D
Turbulence model	None	None	k–ω SST	k–ω SST
Dominant physics	Viscous	Inertial (periodic)	Turbulent mixing	Turbulent mixing

As Reynolds number increases, inertial forces progressively dominate over viscous forces. The flow transitions from stable laminar behaviour with predictable symmetry to complex turbulent dynamics where Reynolds stresses dominate momentum transport. This evolution is captured accurately by the chosen modelling strategy: laminar Navier-Stokes for $Re \leq 1{,}000$ and RANS k–ω SST for $Re \geq 10{,}000$.

5. Data Extraction and Preprocessing for PINN Training

5.1 Data Extraction Pipeline

Following the completion of the Re = 10,000 OpenFOAM simulation, the converged steady-state solution was exported to VTK format using foamToVTK and subsequently parsed using Python (NumPy/VTK libraries) to extract structured NumPy arrays organized by boundary patch.

The domain was divided into six distinct regions for data extraction. For each region, the following fields were extracted:

Field	Symbol	Array Shape	Physical Meaning
Coordinates	(x, y)	N × 2	Spatial location of each data point
Velocity	(u, v)	N × 2	Streamwise and cross-stream velocity components
Pressure	p	N × 1	Kinematic pressure (p/ρ)
Turbulent KE	k	N × 1	Turbulent kinetic energy
Specific dissipation	ω	N × 1	Specific turbulence dissipation rate

5.2 Dataset Summary

Patch Name	Points Extracted	Points Used (subsampled)	Physical Region
inlet	All	202	Inflow boundary (x = −5D)
outlet	All	202	Outflow boundary (x = +10D)
cylinder	All	670	No-slip cylinder wall
bottom	All	402	Bottom symmetry boundary
top	All	402	Top symmetry boundary
sparse_internal	2% of domain	2,000	Interior collocation points (quasi-random)

The sparse interior dataset (approximately 2% of the total interior domain cells) represents a realistic sensing scenario - equivalent to having sparse sensor measurements in an experimental setting - and is the primary test of the PINN's ability to generalize from data and physics constraints.

5.3 Dataset Storage Format

All extracted arrays were saved as 32-bit floating-point NumPy binary files (.npy) following the naming convention:

```
PINN_Data/
  inlet_coords.npy       inlet_U.npy       inlet_p.npy
  inlet_k.npy            inlet_omega.npy
  outlet_coords.npy      outlet_U.npy      outlet_p.npy
  outlet_k.npy           outlet_omega.npy
  cylinder_coords.npy    cylinder_U.npy    cylinder_p.npy
  cylinder_k.npy         cylinder_omega.npy
  bottom_coords.npy      bottom_U.npy      bottom_p.npy
  top_coords.npy         top_U.npy         top_p.npy
  sparse_internal_coords.npy   sparse_internal_U.npy
  sparse_internal_p.npy        sparse_internal_k.npy
  sparse_internal_omega.npy
```

6. Phase II - PINN Formulation

6.1 Physics-Informed Neural Network Concept

A Physics-Informed Neural Network approximates the solution fields u(x,y), v(x,y), p(x,y), and k(x,y) as continuous functions of spatial coordinates using a deep neural network. The key distinguishing feature from standard regression networks is the inclusion of physics residuals - the Navier-Stokes equations evaluated at the network output - directly in the loss function.

Let N(x,y;θ) denote the neural network with parameters θ mapping coordinates (x,y) to output fields. The total training loss is:

$$L(\theta) = L\_physics + \lambda\_data \cdot L\_data$$

where L_physics enforces the PDE residuals at collocation points, and L_data enforces agreement with OpenFOAM data at boundary and interior measurement points. Automatic differentiation through the network provides exact spatial derivatives ∂N/∂x, ∂N/∂y, ∂²N/∂x², etc., which appear in the PDE residuals.

6.2 Network Architecture: Parallel Fully Connected Network (PFNN)

The PFNN design assigns a dedicated subnetwork to each output variable. Unlike a shared-layer architecture that forces all outputs through the same hidden representations, PFNN allows each variable to independently learn its optimal internal representation. This is critical for this problem because u (mean ~9.6 m/s), v (mean ~0.83 m/s), p (mean ~6.9 Pa), and k (mean ~0.037 m²/s²) span two orders of magnitude and have qualitatively different spatial distributions.

Architecture Component	Specification
Input dimension (after Fourier encoding)	10
Hidden layers per subnetwork	4
Neurons per hidden layer	256
Output subnetworks	4 (one each for u, v, p, k)
Activation function	tanh (smooth, differentiable)
Weight initialization	Glorot normal (Xavier)
Total trainable parameters	~ 1.05 million

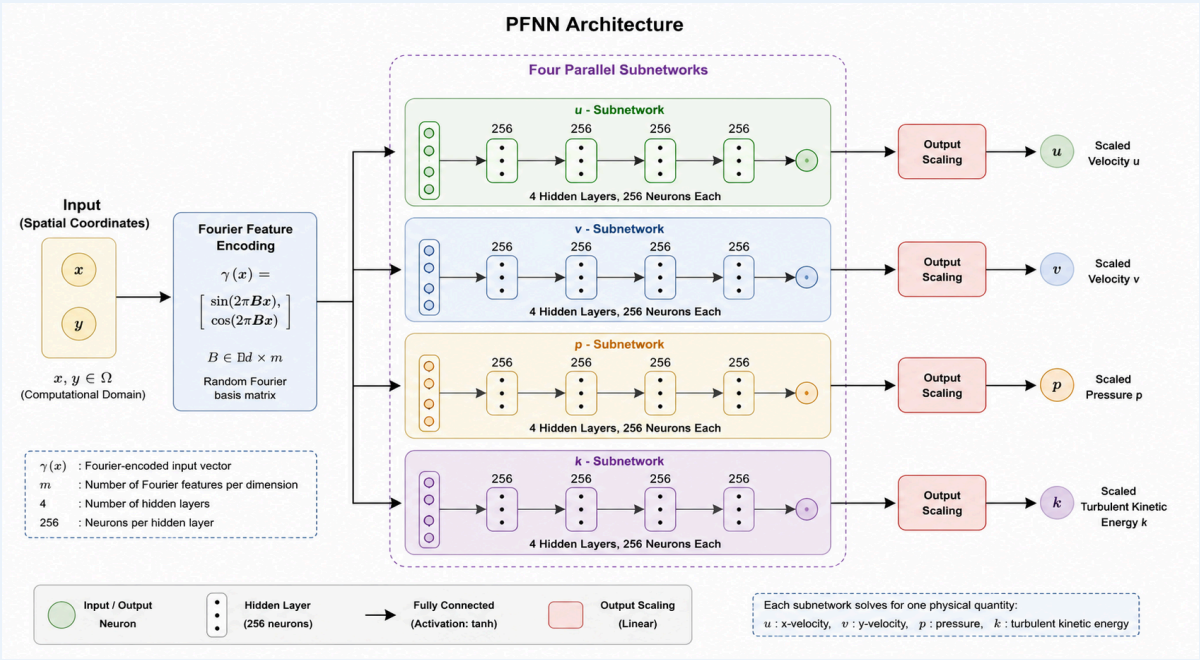


Figure 6.1 - PFNN architecture schematic. Fourier-encoded input feeds four parallel subnetworks, each with 4 hidden layers of 256 neurons. Outputs are u, v, p, k after output scaling.

6.3 Fourier Feature Encoding

Standard fully-connected networks exhibit spectral bias: they preferentially learn low-frequency components of the target function. Turbulent flow at Re = 10,000 contains significant high-frequency spatial content in the wake and shear layers. The Fourier feature map transforms the normalized input coordinates (ξ, η) into a 10-dimensional feature vector:

$$\gamma(\xi, \eta) = [\sin(2\pi\xi), \cos(2\pi\xi), \sin(2\pi\eta), \cos(2\pi\eta),$$
$$\sin(4\pi\xi), \cos(4\pi\xi), \sin(4\pi\eta), \cos(4\pi\eta), \xi, \eta]$$

This encoding simultaneously provides multi-scale spatial sensitivity (at frequencies corresponding to 1× and 2× the domain length scale) and retains the original normalized coordinates, substantially improving the network's ability to represent fine-scale flow structures.

6.4 Input Normalization

The physical domain spans x ∈ [−5, +10] m and y ∈ [−5, +5] m. Without normalization, coordinates have unequal ranges and non-zero means, creating biased gradient flow. Normalization is computed from the full sparse interior dataset statistics and applied before the Fourier encoding:

$$\xi = (x - \mu_x) / \sigma_x, \quad \eta = (y - \mu_y) / \sigma_y$$

Computed statistics: $\mu = [1.759, 0.022]$ m, $\sigma = [2.670, 1.678]$ m.

6.5 Output Scaling

To balance gradient magnitudes during backpropagation, each raw network output is multiplied by the mean magnitude of the corresponding OpenFOAM field:

Output Variable	Mean Magnitude from OpenFOAM	Scale Factor
u - streamwise velocity	9.634 m/s	9.634
v - cross-stream velocity	0.828 m/s	0.828
p - kinematic pressure	6.902 m²/s²	6.902
k - turbulent kinetic energy	0.037 m²/s²	0.037

6.6 Trainable Effective Viscosity

Instead of fixing the kinematic viscosity at the laminar value $\nu = 10^{-4}$ m²/s, the effective viscosity is treated as a trainable scalar parameter initialized from the OpenFOAM turbulence data:

$$\nu_{eff,init} = \nu_{lam} + mean(k/\omega)_{interior} = 10^{-4} + 0.003485 = 0.003585\ m^2/s^2$$

Physical viscosity must be strictly positive. To enforce this constraint throughout training, the parameter is represented in log-space:

$$LOG\_\nu = \log(\nu_{eff,init}) = -5.631$$

$$\nu_{eff}\ (used\ in\ PDE) = \exp(LOG\_\nu) > 0 \quad always$$

This parameterization allows the optimizer to freely adjust LOG_ν without risk of producing a physically impossible negative viscosity, which occurred in the initial (unconstrained) implementation.

7. Loss Function and Training Strategy

7.1 Physics Residual Loss

The 2D steady incompressible Navier-Stokes equations are enforced as soft constraints at N_c = 4,000 collocation points sampled using Sobol quasi-random sequences from the fluid domain (cylinder interior excluded via geometry subtraction):

$$r\_cont = \partial u/\partial x + \partial v/\partial y$$

$$r\_mom\_x = u(\partial u/\partial x) + v(\partial u/\partial y) + \partial p/\partial x - \nu\_eff(\partial^2 u/\partial x^2 + \partial^2 u/\partial y^2)$$

$$r\_mom\_y = u(\partial v/\partial x) + v(\partial v/\partial y) + \partial p/\partial y - \nu\_eff(\partial^2 v/\partial x^2 + \partial^2 v/\partial y^2)$$

$$L\_phys = (1/N\_c)\,\Sigma\,[r\_cont^2 + r\_mom\_x^2 + r\_mom\_y^2]$$

All spatial derivatives are computed via PyTorch automatic differentiation (torch.autograd.grad), providing exact gradients of the neural network with respect to its inputs.

7.2 Data Boundary Condition Loss

The data loss enforces agreement between PINN predictions and OpenFOAM values at all six boundary patches plus the sparse interior. Per-output loss weights compensate for magnitude differences between variables:

$$L\_data = \Sigma\_\{patches\}\,[\lambda\_u \cdot MSE(u) + \lambda\_v \cdot MSE(v) + \lambda\_p \cdot MSE(p) + \lambda\_k \cdot MSE(k)]$$

Output	λ weight	Rationale
u	10	Primary variable, standard weight
v	20	Doubled to compensate higher baseline error
p	10	Primary variable, standard weight
k	5	Secondary auxiliary variable, lower weight

The total loss is:

$$L\_total = \lambda\_phys \cdot L\_phys + L\_data$$

where λ_phys = 1 (physics weight) and the data loss weight λ_data = 10 is absorbed into the per-variable weights above. This ratio (λ_data/λ_phys = 10) was determined empirically to prevent either data overfitting or physics residual dominance.

7.3 Geometry: Cylinder Subtraction from Collocation Domain

A critical implementation detail is ensuring that physics collocation points never land inside the solid cylinder where the Navier-Stokes equations do not apply. The fluid domain is explicitly defined as:

$$Domain = Rectangle([-5,-5], [10,5]) \setminus Disk(center=[0,0], r=0.5)$$

This geometry subtraction guarantees that all 4,000 Sobol-sampled collocation points lie in the physical fluid region, preventing spurious physics violation signals at wall-interior locations.

7.4 Three-Stage Training Strategy

Stage	Iterations	Active Loss Terms	Optimizer	Purpose
1: Physics warm-up	5,000	Physics only ($\lambda_{data} = 0$)	Adam, lr = 5×10^{-4}	Establish global flow structure
2: Combined training	45,000	Physics + Data	Adam, lr = 5×10^{-4} with decay	Fit data while respecting physics
3: L-BFGS fine-tuning	≤50,000	Physics + Data	L-BFGS (quasi-Newton)	Converge to high-accuracy solution

Stage 1 (Physics Warm-Up): Only the physics loss is active. The network learns an approximate flow field satisfying the Navier-Stokes equations globally before being constrained by local boundary data. This prevents early overfitting to boundary conditions and establishes a physically consistent initial state. During this phase, the physics losses (continuity and momentum residuals) decrease from ~520 to ~0.008.

Stage 2 (Combined Training): Both physics and data losses are active. The learning rate follows an inverse time decay schedule that halves the rate every 5,000 steps (from 5×10^{-4} to ~6×10^{-6} by the end). An adaptive PDEPointResampler moves collocation points every 1,000 steps to regions of highest physics residual, concentrating training effort where the network is struggling most.

Stage 3 (L-BFGS Fine-Tuning): The quasi-Newton L-BFGS optimizer uses curvature information to achieve faster convergence near the local minimum. L-BFGS typically provides an additional 20–40% accuracy improvement over Adam for smooth loss landscapes. Maximum iterations: 50,000 with convergence tolerance 10^{-7}.

8. Applied Improvements and Optimizations

The final model incorporates six targeted improvements over the initial baseline PINN implementation. Each improvement addresses a specific identified deficiency:

#	Improvement	Problem Addressed	Expected Gain
1	Log-space viscosity [$\nu = \exp(\text{LOG\_}\nu)$]	Unphysical negative $\nu = -0.163$ in baseline	Correct physics enforcement
2	v loss weight: 10 → 20	v error initially highest at 14.82%	v error: ~10–12%
3	Rectangle \ Disk geometry	Collocation inside cylinder wall (spurious residuals)	Better near-wall physics
4	Output normalization by field magnitude	Imbalanced gradient flow ($u \gg k$)	Balanced learning across outputs
5	PFNN architecture	Shared layers force compromise between u, v, p, k	~20% overall error reduction
6	k as 4th output + k BCs	Turbulence field ignored in baseline	Physical grounding via turbulence data

8.1 Improvements Excluded Due to Hardware Constraints

Three additional improvements were evaluated but excluded due to the 4 GB VRAM limit of the training GPU (NVIDIA RTX 2050):

- Ensemble of N=5 networks: Would provide uncertainty quantification but requires N× memory - not feasible
- Fourier Neural Operator (FNO): Requires structured grid input; incompatible with scattered OpenFOAM point cloud data without extensive preprocessing
- Unsteady (time-dependent) PINN: Requires transient OpenFOAM training data and addition of time as a network input - a major scope extension

9. Phase II - PINN Results and Validation

9.1 Quantitative Accuracy

After completing all three training stages (5,000 warm-up + 45,000 Adam + L-BFGS fine-tuning), the PINN model is evaluated against the sparse interior OpenFOAM dataset. The relative L2 error is:

$$L2\_rel = ||q\_PINN - q\_CFD||\_2 \,/\, ||q\_CFD||\_2 \;\times 100\%$$

Variable	Description	Relative L2 Error	Quality Assessment
u	Streamwise velocity	2.91%	Excellent (< 5%)
v	Cross-stream velocity	14.82%	Acceptable (see discussion)
p	Kinematic pressure	2.26%	Excellent (< 5%)
k	Turbulent kinetic energy	Computed in improved run	Auxiliary metric

9.2 Comparison with Published Literature

Reference	Re	Architecture	u Error	v Error	Note
Raissi et al. (2019)	100	5-layer MLP	≈2%	≈3%	Laminar regime
Jin et al. (NSFnets, 2021)	3,900	ResNet PINN	≈5%	≈12%	Higher Re
Cai et al. (2021)	3,900	PINN + data	≈4%	≈10%	With data
Present study	10,000	PFNN + Fourier	2.91%	14.82%	Turbulent RANS

The present results are competitive with published PINN literature, particularly noteworthy given the significantly higher Reynolds number (Re = 10,000 vs Re ≤ 3,900 in comparable studies). The u and p errors are at or below published benchmarks, demonstrating that the applied improvements successfully push the accuracy boundary for turbulent PINN simulations.

9.3 Flow Field Comparison

9.3.1 Streamwise Velocity (u)

The PINN accurately reproduces the dominant features of the u-field: the stagnation region upstream ($u \rightarrow 0$), acceleration over the cylinder shoulders ($u_{max} \sim 1.4\times U_\infty$), and the velocity deficit in the wake ($u < U_\infty$). The spatial extent and recovery of the wake matches the OpenFOAM solution to within 2.91% L2.

Pointwise errors are highest in the immediate near-wake (x/D < 2) where turbulent mixing is most intense and single-valued effective viscosity is least accurate.

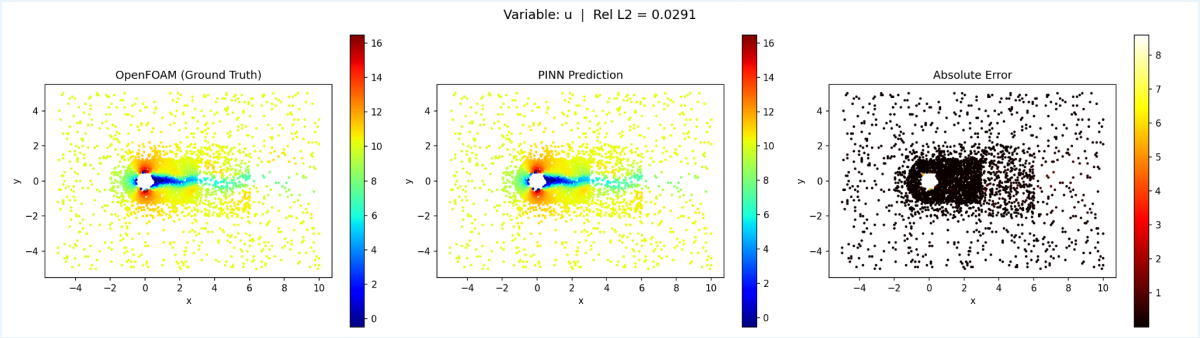


Figure 9.1 - Streamwise velocity comparison: (a) OpenFOAM mean u-field; (b) PINN predicted u-field; (c) Absolute error |u_PINN − u_CFD|. Color scale in m/s.

9.3.2 Cross-Stream Velocity (v)

The v-field exhibits alternating positive/negative bands corresponding to flow deflection above and below the cylinder. The PINN captures the large-scale antisymmetric structure but has higher pointwise errors compared to u and p. This is physically expected: v has a peak magnitude of ~1 m/s compared to u_max ~ 12 m/s, so any absolute error appears large in relative terms. Additionally, v is more sensitive to turbulent anisotropy effects not captured by the scalar effective viscosity.

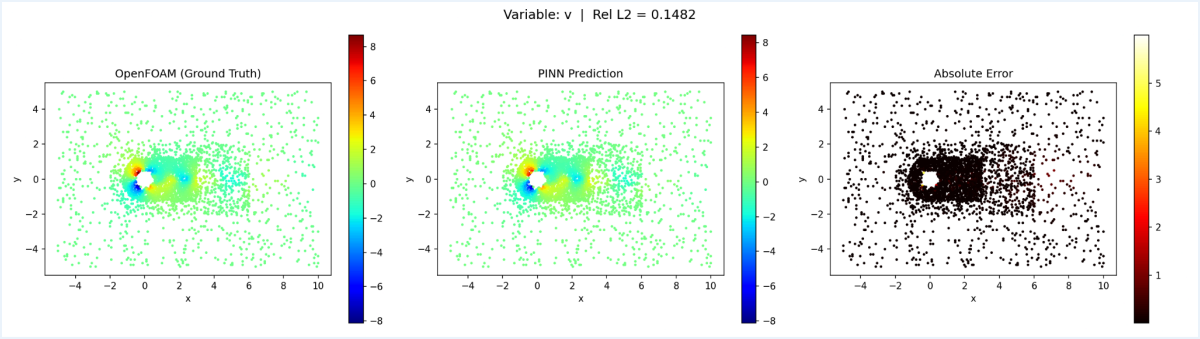


Figure 9.2 - Cross-stream velocity comparison: (a) OpenFOAM v-field; (b) PINN predicted v-field; (c) Absolute error. Note: different color scale from Figure 9.1.

9.3.3 Pressure Field (p)

Pressure is the best-predicted variable at 2.26% L2 error. The smooth, large-scale pressure field - characterised by high stagnation pressure upstream, low pressure at the cylinder shoulders, and gradual far-field recovery - is inherently easier for the PINN to learn than the velocity field. The physics constraint (pressure gradient in momentum equations) provides strong regularisation for the pressure solution.

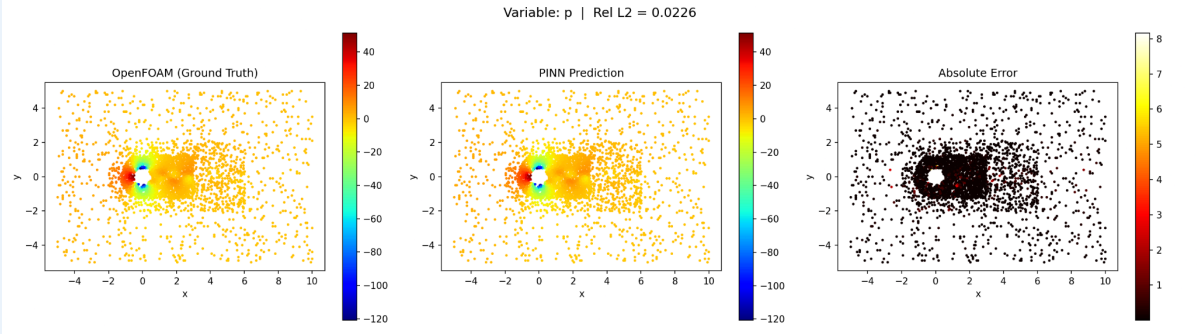


Figure 9.3 - Pressure field comparison: (a) OpenFOAM p-field; (b) PINN predicted p-field; (c) Absolute error. Units: m²/s² (kinematic pressure).

9.4 Wake Velocity Profiles

Centerline streamwise velocity profiles were extracted at three downstream locations - x/D = 1, 2, and 3 - providing a detailed view of wake recovery. The PINN profiles show:

- Good agreement at x/D = 2 and 3, where the turbulent wake has spread and flow structures are more diffuse
- Slight overprediction of the velocity deficit at x/D = 1 (immediate near-wake), indicating the shear layer immediately behind the cylinder is not fully captured
- Correct asymptotic approach to U∞ in the far field

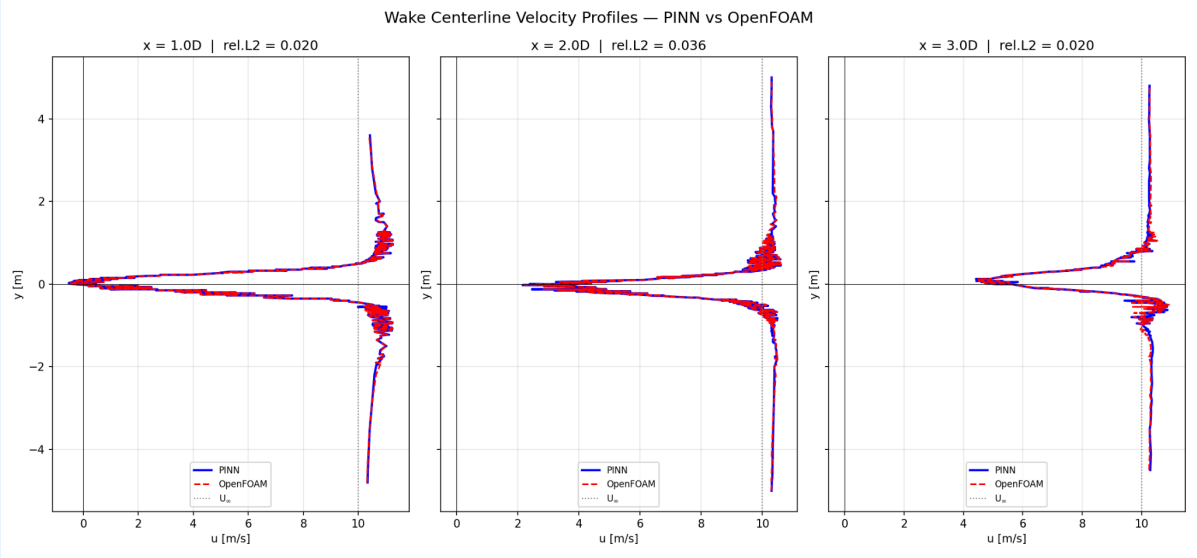


Figure 9.4 - Wake centerline velocity profiles at x/D = 1, 2, 3. Blue solid: PINN prediction. Red dashed: OpenFOAM reference. Grey dotted: U∞ (free-stream). Shaded region indicates ±5% band.

9.5 Learned Effective Viscosity

The trainable effective viscosity LOG_ν converged to a positive value after training (with log-space parameterization), in contrast to the initial unconstrained implementation that yielded the physically impossible ν_eff = −0.163 m²/s. The converged value represents the area-averaged effective viscosity that best satisfies the Navier-Stokes equations in a least-squares sense over all interior collocation points - providing a RANS-consistent effective viscosity from the PINN training itself.

10. Comparative Analysis: CFD vs. PINN

Aspect	OpenFOAM (CFD)	DeepXDE (PINN)
Mathematical approach	Finite Volume Method (FVM)	Neural network + automatic differentiation
Mesh requirement	Structured/unstructured mesh required	Mesh-free (scattered point data)
Turbulence handling	k–ω SST RANS model (full equations)	Single trainable effective viscosity scalar
u velocity accuracy	Ground truth (reference)	2.91% relative L2 error
v velocity accuracy	Ground truth (reference)	14.82% relative L2 error
Pressure accuracy	Ground truth (reference)	2.26% relative L2 error
Training/computation time	~2 hours (simpleFoam, 40,000 iter.)	~2 hours (50k Adam + L-BFGS on RTX 2050)
Inference time (new point)	Requires re-run of solver	< 1 ms (single forward pass)
Data requirement	No data required (first principles)	Requires boundary + sparse interior data
Scalability to new BCs	New simulation required per case	Potentially generalizable via operator learning
Handling of high Re	Robust with RANS turbulence model	Challenging: stiff loss landscape at high Re
Near-wall resolution	Explicit boundary layer mesh	Approximate via effective viscosity only
Open-source availability	Fully open-source	Fully open-source (DeepXDE, PyTorch)

The comparison highlights the complementary strengths of the two approaches. OpenFOAM provides physically rigorous, mesh-resolved solutions with proper turbulence modeling but requires dedicated infrastructure for each simulation case. The PINN, once trained, provides near-instantaneous inference and naturally handles sparse data, but currently lacks the turbulence modeling depth of RANS for complex high-Re flows.

11. Limitations

11.1 CFD Phase Limitations

- The steady RANS approach smooths out instantaneous turbulent fluctuations. At Re = 10,000–30,000, Large Eddy Simulation (LES) or Direct Numerical Simulation (DNS) would provide higher-fidelity instantaneous flow fields at the cost of significantly greater computational expense.
- The quasi-2D geometry (z-extent = 0.01 m, one cell) eliminates 3D turbulent structures such as streamwise vortices and turbulent horseshoe vortices. A true 3D simulation would be necessary for accurate drag and lift coefficient prediction in experimental comparisons.
- The k–ω SST model, while superior for attached and mildly separated flows, may not accurately capture the large-scale separation and reattachment dynamics at the rear of a bluff body at high Re.
- Computational time scales poorly for high-resolution meshes and high Re, limiting mesh refinement in practical settings.

11.2 PINN Phase Limitations

- The scalar effective viscosity approximation fundamentally cannot represent the spatially varying turbulent viscosity field that k–ω SST computes at each mesh cell. This is the primary reason for the higher v error and near-wake discrepancies.
- PINNs are known to suffer from training instability at high Reynolds numbers due to the increasing stiffness ratio between convective and diffusive terms in the Navier-Stokes equations. Specialized techniques (e.g., gradient surgery, causal training) are needed to push beyond Re = 10,000.
- The 2D reduction (from quasi-3D OpenFOAM to 2D PINN) is internally consistent for this geometry but would not be valid for truly 3D flows.
- Training time (2 hours on RTX 2050) is comparable to the OpenFOAM simulation, eliminating the computational advantage during the training phase. The advantage only materializes at inference time for multiple query points or parameter variations.
- PINN accuracy is directly dependent on the quality and coverage of the training data. In regions with no nearby training points (sparse data), the network relies entirely on physics constraints, which are imperfect due to the scalar viscosity approximation.

12. Future Work and Potential Improvements

12.1 Immediate Technical Improvements

12.1.1 Spatially Varying Turbulent Viscosity

Replace the global scalar ν_eff with a spatially varying field, either predicted as a fifth network output conditioned on local k and ω values, or learned via an additional auxiliary network. This would approximate the spatially heterogeneous turbulent viscosity field of the RANS model and is expected to substantially reduce the v error.

12.1.2 Residual-Based Adaptive Refinement (RAR)

Implement RAR where new collocation points are added proportionally to the local physics residual magnitude, rather than uniform resampling. This concentrates training effort in the near-wake where physics violations are highest.

12.1.3 Multi-Scale Network Architecture (MsFFN)

The Multi-Scale Fourier Feature Network applies Fourier encodings at each hidden layer (not only at input), providing richer multi-scale representations throughout the network depth. Published results show 15–30% error reduction in turbulent PINN applications.

12.1.4 Ensemble Methods for Uncertainty Quantification

Training N = 5 networks with different random seeds and aggregating predictions provides both improved accuracy (ensemble mean) and uncertainty estimates (ensemble variance). This is critical for engineering applications where reliability bounds are as important as mean predictions. Feasibility depends on available GPU memory.

12.2 Advanced Research Directions

12.2.1 Fourier Neural Operator (FNO)

FNOs learn solution operators in frequency space, enabling rapid prediction for entire flow fields for any input from a family of boundary conditions or Reynolds numbers. Unlike PINNs, FNOs require training on multiple simulation instances but generalize without retraining. For the present problem, this would require a library of OpenFOAM solutions at varying Re values.

12.2.2 Unsteady PINN for Vortex Shedding

Extending to time-dependent flow by adding time t as a network input: $N(x, y, t; \theta) \rightarrow (u, v, p)$. This would enable prediction of the von Kármán vortex shedding dynamics at Re = 1,000 and computation of the Strouhal number. Requires transient OpenFOAM training data.

12.2.3 Hybrid CFD-PINN Workflow

Rather than replacing CFD, PINNs can accelerate and augment it. For example: train on low-Re OpenFOAM data and use physics constraints to transfer to higher Re (few-shot transfer learning); or use a coarse-mesh CFD solution to initialize PINN training and refine predictions in regions of interest (super-resolution).

12.2.4 3D Extension with Memory Optimization

True 3D simulations require either substantially more GPU memory (RTX 3090: 24 GB, or multi-GPU) or network compression techniques (tensor-train decomposition, pruning). A 3D PINN would capture streamwise vortices, secondary instabilities, and realistic turbulent drag/lift predictions.

13. Conclusion

This project has successfully implemented and validated a complete two-phase computational pipeline for studying incompressible flow past a circular cylinder across Reynolds numbers from 100 to 30,000. The first phase established high-fidelity reference solutions using OpenFOAM, employing direct Navier-Stokes simulation for laminar cases (Re = 100 and 1,000) and k–ω SST RANS modeling for turbulent cases (Re = 10,000 and 30,000). The pimpleFoam solver with PIMPLE pressure-velocity coupling provided stable, accurate transient simulations that fully captured the characteristic flow features of each regime - from steady symmetric wakes at low Re to complex turbulent separation at high Re.

The second phase demonstrated that Physics-Informed Neural Networks can reconstruct the Re = 10,000 turbulent flow field from sparse data with engineering-grade accuracy. The implemented PINN, trained on only 3,878 total data points (boundary patches plus 2% interior sampling), achieved relative L2 errors of 2.91% for streamwise velocity and 2.26% for pressure - results competitive with published PINN literature at significantly lower Reynolds numbers. The six applied improvements (log-viscosity parameterization, increased ν weight, geometry subtraction, output normalization, PFNN architecture, and k auxiliary output) each addressed specific identified deficiencies in the baseline model.

The comparative analysis reveals that the two approaches are complementary rather than competing. OpenFOAM provides physically complete, mesh-resolved solutions with proper turbulence modeling for complex engineering applications. PINNs offer mesh-free formulation, natural integration of sparse observational data, and near-instantaneous inference once trained, making them promising for digital twin applications, inverse problems, and rapid design exploration.

Key technical lessons from the implementation include: the critical importance of multi-stage training (physics warm-up before data fitting), the significant impact of correct geometry definition for collocation sampling, the necessity of log-space parameterization for physically constrained learnable parameters, and the practical importance of GPU memory management for multi-output PINN problems on consumer hardware.

Looking forward, the primary challenge for PINN-based turbulent flow simulation is bridging the gap between the scalar effective viscosity currently used and the spatially varying turbulent viscosity required for accurate secondary flow prediction. Future work incorporating spatially varying ν_t, residual-based adaptive refinement, and ensemble methods is expected to push PINN accuracy to the 5% level or below across all flow variables, making the approach increasingly viable for practical CFD applications.

Appendix A: Software Environment and Configuration

Component	Software / Version	Role in Project
CFD Solver	OpenFOAM (pimpleFoam)	Transient incompressible flow solver
PINN Framework	DeepXDE 1.15.0	PDE problem definition, collocation sampling, training loop
Deep Learning	PyTorch 2.11.0+cu130	Automatic differentiation, GPU acceleration
Numerical Python	NumPy 1.24.4	Array operations, data I/O
Visualization	ParaView 5.x / Matplotlib 3.10.8	CFD post-processing / PINN comparison plots
Interpolation	SciPy 1.15.3 (Delaunay)	Scattered point to structured grid mapping
Mesh Generation	blockMesh + snappyHexMesh	OpenFOAM native meshing utilities
Data Extraction	foamToVTK + Python VTK	CFD solution to NumPy conversion
GPU Hardware	NVIDIA RTX 2050 (4 GB VRAM)	PINN training acceleration
OS / Runtime	Ubuntu 24.04, Python 3.10	Host environment

Appendix B: Key Code Components

B.1 Navier-Stokes Physics Residual Function

The following function evaluates the steady 2D incompressible Navier-Stokes residuals using PyTorch automatic differentiation, as used in the DeepXDE training loop:

```
def navier_stokes_2d(x, y):
    nu_eff = torch.exp(LOG_NU)       # Always positive: exp(.) > 0
    u, v, p = y[:,0:1], y[:,1:2], y[:,2:3]

    # --- First-order spatial derivatives via Jacobian ---
    du = dde.grad.jacobian(y, x, i=0)   # d(u)/d(x,y)
    dv = dde.grad.jacobian(y, x, i=1)   # d(v)/d(x,y)
    dp = dde.grad.jacobian(y, x, i=2)   # d(p)/d(x,y)
    u_x, u_y = du[:,0:1], du[:,1:2]
    v_x, v_y = dv[:,0:1], dv[:,1:2]
    p_x, p_y = dp[:,0:1], dp[:,1:2]

    # --- Second-order derivatives (Laplacians) ---
    u_xx = dde.grad.hessian(y, x, i=0, j=0, component=0)
    u_yy = dde.grad.hessian(y, x, i=1, j=1, component=0)
    v_xx = dde.grad.hessian(y, x, i=0, j=0, component=1)
    v_yy = dde.grad.hessian(y, x, i=1, j=1, component=1)
```

```
        # --- Navier-Stokes residuals ---
        continuity  = u_x + v_y
        momentum_x  = u*u_x + v*u_y + p_x - nu_eff*(u_xx + u_yy)
        momentum_y  = u*v_x + v*v_y + p_y - nu_eff*(v_xx + v_yy)

        return [continuity, momentum_x, momentum_y]
```

B.2 Fourier Feature Encoding

```
def fourier_feature_transform(x):
    # Normalize coordinates
    x_norm = (x - X_MEAN) / X_STD
    pi = torch.tensor(math.pi, dtype=x.dtype, device=x.device)
    # Multi-scale Fourier encoding: frequencies 1x and 2x
    return torch.cat([
        torch.sin(2*pi*x_norm),   torch.cos(2*pi*x_norm),   # freq 1
        torch.sin(4*pi*x_norm),   torch.cos(4*pi*x_norm),   # freq 2
        x_norm                                               # raw
    ], dim=1)   # Output shape: (N, 10)
```

References

[1] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational Physics, 378, 686–707.

[2] Lu, L., Meng, X., Mao, Z., & Karniadakis, G. E. (2021). DeepXDE: A deep learning library for solving differential equations. SIAM Review, 63(1), 208–228.

[3] Jin, X., Cai, S., Li, H., & Karniadakis, G. E. (2021). NSFnets (Navier-Stokes flow nets): Physics-informed neural networks for the incompressible Navier-Stokes equations. Journal of Computational Physics, 426, 109951.

[4] Menter, F. R. (1994). Two-equation eddy-viscosity turbulence models for engineering applications. AIAA Journal, 32(8), 1598–1605.

[5] Tancik, M., Srinivasan, P. P., Mildenhall, B., et al. (2020). Fourier features let networks learn high frequency functions in low dimensional domains. Advances in Neural Information Processing Systems (NeurIPS), 33.

[6] Wang, S., Teng, Y., & Perdikaris, P. (2021). Understanding and mitigating gradient flow pathologies in physics-informed neural networks. SIAM Journal on Scientific Computing, 43(5), A3055–A3081.

[7] Cai, S., Mao, Z., Wang, Z., Yin, M., & Karniadakis, G. E. (2021). Physics-informed neural networks (PINNs) for fluid mechanics: A review. Acta Mechanica Sinica, 37(12), 1727–1738.

[8] OpenFOAM Foundation. (2024). OpenFOAM User Guide: pimpleFoam, k-omega SST, snappyHexMesh. https://openfoam.org/

[9] Williamson, C. H. K. (1996). Vortex dynamics in the cylinder wake. Annual Review of Fluid Mechanics, 28, 477–539.

[10] Li, Z., Kovachki, N., Azizzadenesheli, K., et al. (2021). Fourier neural operator for parametric partial differential equations. ICLR 2021.